



Redictado Orientación a Objetos 2

Refactoring

Fecha de última edición: 5 septiembre 2025

Ejercicio -1

Estudie en detalle los Code Smells y los Refactoring del libro “Refactoring - Improving The Design of Existing Code” de Martin Fowler y una con flechas los code smells y los refactorings a los que se relacionan.

- Duplicate Code
- Large Class
- Long Method
- Data Class
- Feature Envy
- Long Parameter List
- Switch Statements
- Rename Variable
- Rename Method
- Extract Method
- Move Method
- Replace Conditional with Polymorphism
- Replace temp with query
- Pull Up Field
- Pull up Method

Ejercicio 0

Estudie los refactorings listados del libro “Refactoring - Improving The Design of Existing Code” de Martin Fowler y liste el conjunto de precondiciones para cada uno:

- Rename Variable/field (pág. 14)
- Rename Method
- Extract Method
- Move Method
- Replace Conditional with Polymorphism
- Replace temp with query
- Pull Up Field
- Pull up Method

Ejercicio 1: Algo huele mal

Indique qué malos olores se presentan en los siguientes ejemplos.

1.1 - Protocolo de Cliente

La clase `Cliente` tiene el siguiente protocolo. ¿Cómo puede mejorarlo?

```
/**
 * Retorna el límite de crédito del cliente.
 */
protected double lmtCrdt() {...}

/**
 * Retorna el monto facturado al cliente desde la fecha f1 a la fecha f2.
 */
protected double mtFcE(LocalDate f1, LocalDate f2) {...}

/**
 * Retorna el monto cobrado al cliente desde la fecha f1 a la fecha f2.
 */
protected double mtCbE(LocalDate f1, LocalDate f2) {...}
```

1.2 - Participación en proyectos

Al revisar el siguiente diseño inicial (Figura 1), se decidió realizar un cambio para evitar lo que se consideraba un mal olor. El diseño modificado se muestra en la Figura 2. Indique qué tipo de cambio se realizó y si lo considera apropiado. Justifique su respuesta.

Diseño inicial:

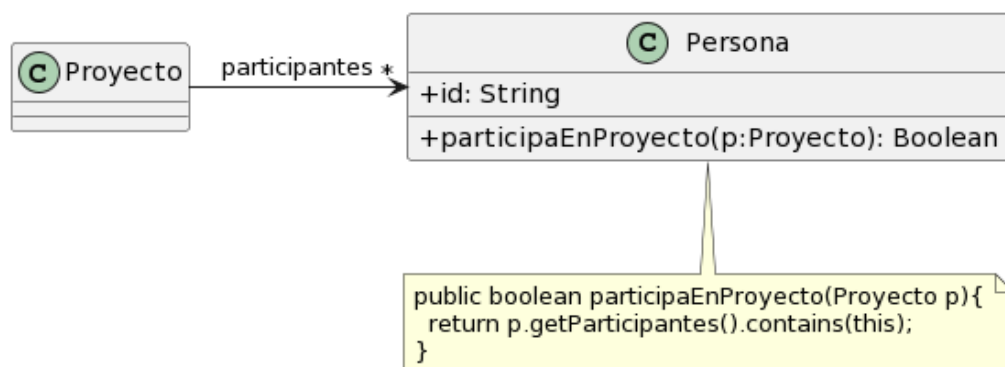


Figura 1: Diagrama de clases del diseño inicial.

Diseño revisado:

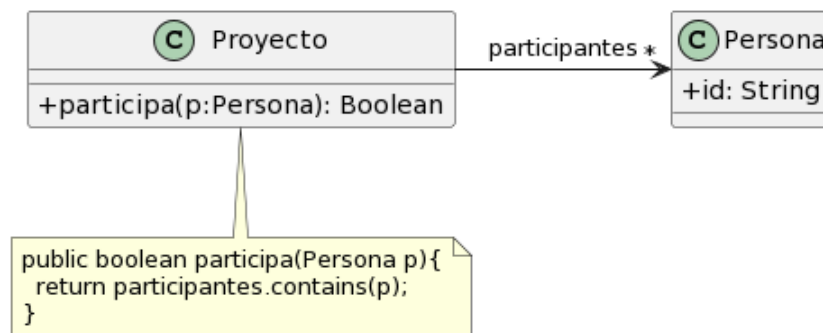


Figura 2: Diagrama de clases modificado.

1.3 - Cálculos

Analice el código que se muestra a continuación. Indique cuáles podrían ser bugs vs. code smells, y cuál sería la forma de corregir cada uno.

```

public void imprimirValores() {
    int totalEdades = 0;
    double promedioEdades = 0;
    double totalSalarios = 0;

    for (Empleado empleado : personal) {
        totalEdades = totalEdades + empleado.getEdad();
        totalSalarios = totalSalarios + empleado.getSalario();
    }
    promedioEdades = totalEdades / personal.size();

    String message = String.format("El promedio de las edades es %s y el
total de salarios es %s", promedioEdades, totalSalarios);

    System.out.println(message);
}
    
```

Ejercicio 2

Para cada una de las siguientes situaciones, realice en forma iterativa los siguientes pasos:

- I. indique el mal olor,
- II. indique el refactoring que lo corrige,
- III. aplique el refactoring, mostrando el resultado final (código y/o diseño según corresponda).

Si vuelve a encontrar un mal olor, retorne al paso "I".



2.1 - Empleados

```
public class EmpleadoTemporario {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    public double horasTrabajadas = 0;
    public int cantidadHijos = 0;
    // .....

    public double sueldo() {
        return this.sueldoBasico + (this.horasTrabajadas * 500) +
            (this.cantidadHijos * 1000) - (this.sueldoBasico * 0.13);
    }
}

public class EmpleadoPlanta {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    public int cantidadHijos = 0;
    // .....

    public double sueldo() {
        return this.sueldoBasico + (this.cantidadHijos * 2000) -
            (this.sueldoBasico * 0.13);
    }
}

public class EmpleadoPasante {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    // .....

    public double sueldo() {
        return this.sueldoBasico - (this.sueldoBasico * 0.13);
    }
}
```

2.2 - Juego

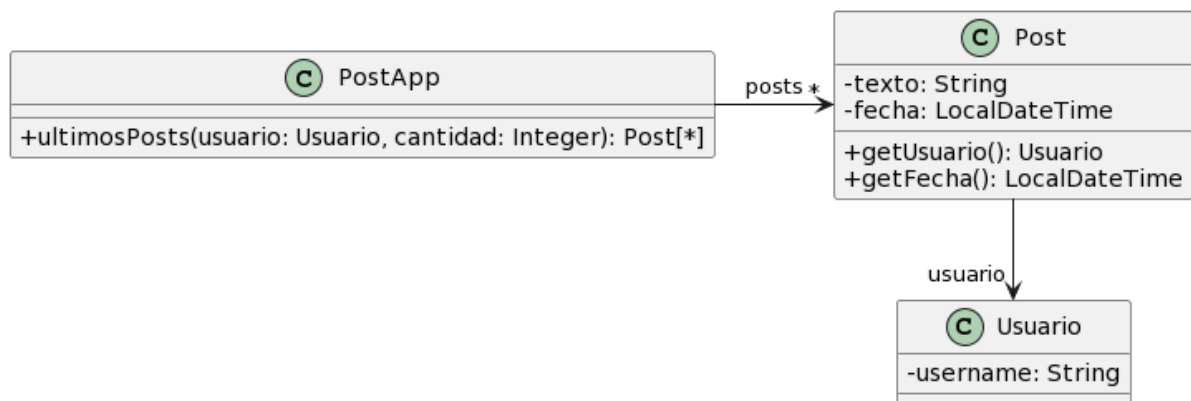
```
public class Juego {
    // .....

    public void incrementar(Jugador j) {
        j.puntuacion = j.puntuacion + 100;
    }
}
```



```
}  
  
public void decrementar(Jugador j) {  
    j.puntuacion = j.puntuacion - 50;  
}  
  
}  
  
public class Jugador {  
    public String nombre;  
    public String apellido;  
    public int puntuacion = 0;  
}
```

2.3 - Publicaciones



```
/**  
 * Retorna los últimos N posts que no pertenecen al usuario user  
 */  
public List<Post> ultimosPosts(Usuario user, int cantidad) {  
    List<Post> postsOtrosUsuarios = new ArrayList<Post>();  
    for (Post post : this.posts) {  
        if (!post.getUsuario().equals(user)) {  
            postsOtrosUsuarios.add(post);  
        }  
    }  
  
    // ordena los posts por fecha  
    for (int i = 0; i < postsOtrosUsuarios.size(); i++) {  
        int masNuevo = i;  
        for (int j = i + 1; j < postsOtrosUsuarios.size(); j++) {  
            if (postsOtrosUsuarios.get(j).getFecha().isAfter(  
                postsOtrosUsuarios.get(masNuevo).getFecha())) {  
                masNuevo = j;  
            }  
        }  
        Post unPost = postsOtrosUsuarios.set(i,
```

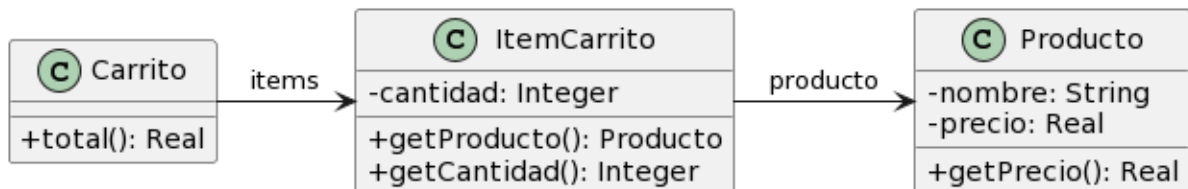
```

        postsOtrosUsuarios.get(masNuevo));
        postsOtrosUsuarios.set(masNuevo, unPost);
    }

    List<Post> ultimosPosts = new ArrayList<Post>();
    int index = 0;
    Iterator<Post> postIterator = postsOtrosUsuarios.iterator();
    while (postIterator.hasNext() && index < cantidad) {
        ultimosPosts.add(postIterator.next());
    }
    return ultimosPosts;
}

```

2.4 - Carrito de compras



```

public class Producto {
    private String nombre;
    private double precio;

    public double getPrecio() {
        return this.precio;
    }
}

public class ItemCarrito {
    private Producto producto;
    private int cantidad;

    public Producto getProducto() {
        return this.producto;
    }

    public int getCantidad() {
        return this.cantidad;
    }
}

public class Carrito {
    private List<ItemCarrito> items;

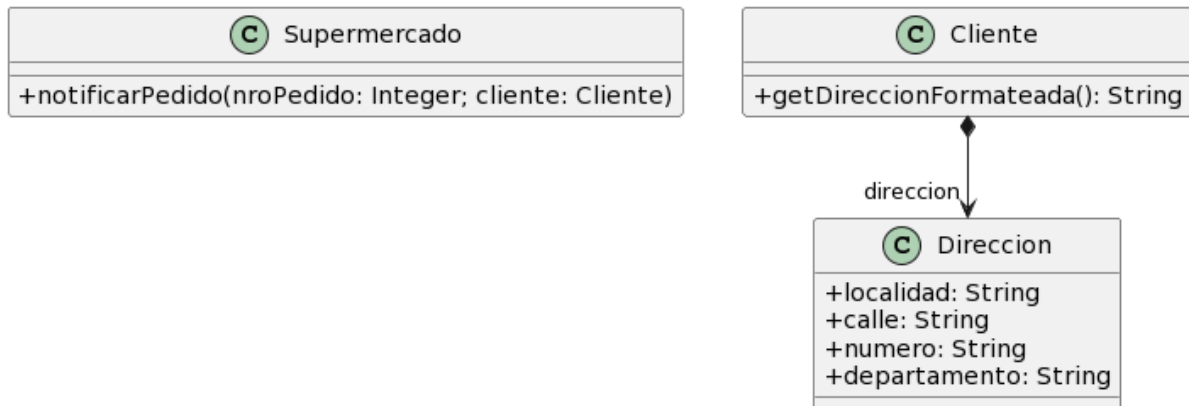
    public double total() {

```



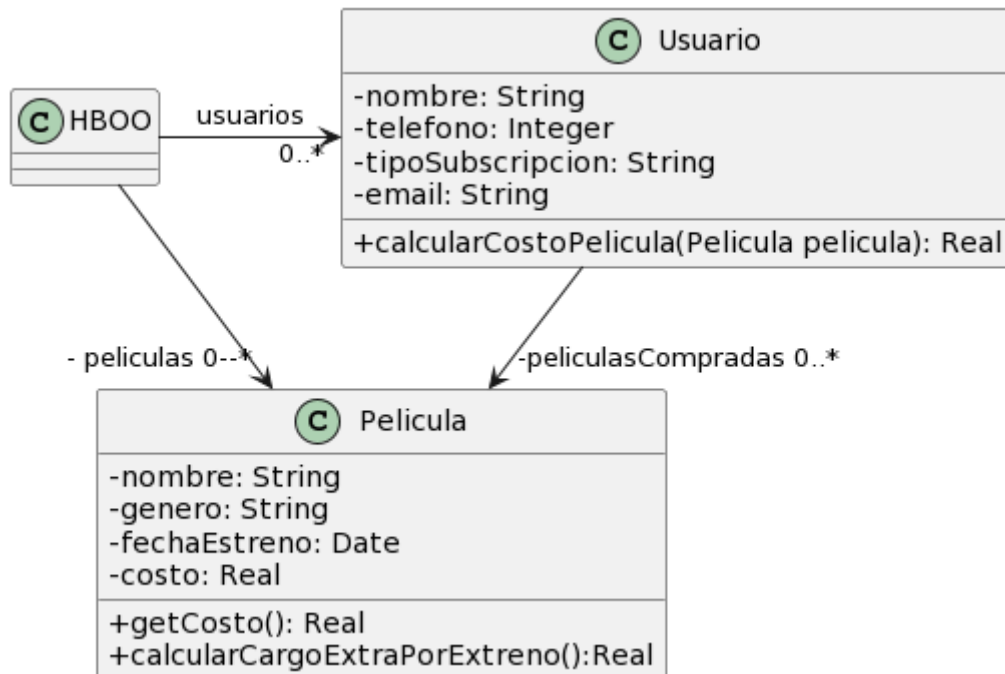
```
        return this.items.stream().mapToDouble(item -> item.getProducto()  
            .getPrecio() * item.getCantidad()).sum();  
    }  
}
```

2.5 - Envío de pedidos



```
public class Supermercado {  
    public void notificarPedido(long nroPedido, Cliente cliente) {  
        String notificacion = MessageFormat.format(  
            "Estimado cliente, se le informa que hemos recibido su pedido  
            con número {0}, el cual será enviado a la dirección {1}",  
            new Object[] { nroPedido, cliente.getDireccionFormateada() }  
        );  
  
        // lo imprimimos en pantalla, podría ser un mail, SMS, etc..  
        System.out.println(notificacion);  
    }  
}  
  
public class Cliente {  
    public String getDireccionFormateada() {  
        return this.direccion.getLocalidad() + ", " +  
            this.direccion.getCalle() + ", " +  
            this.direccion.getNumero() + ", " +  
            this.direccion.getDepartamento();  
    }  
}
```

2.6 - Películas



```

public class Usuario {
    String tipoSubscripcion;
    // ...

    public void setTipoSubscripcion(String unTipo) {
        this.tipoSubscripcion = unTipo;
    }

    public double calcularCostoPelicula(Pelicula pelicula) {
        double costo = 0;
        if (tipoSubscripcion == "Basico") {
            costo = pelicula.getCosto() +
                pelicula.calcularCargoExtraPorEstreno();
        }
        else if (tipoSubscripcion == "Familia") {
            costo = (pelicula.getCosto() +
                pelicula.calcularCargoExtraPorEstreno()) * 0.90;
        }
        else if (tipoSubscripcion == "Plus") {
            costo = pelicula.getCosto();
        }
        else if (tipoSubscripcion == "Premium") {
            costo = pelicula.getCosto() * 0.75;
        }
    }
}
    
```



```
        return costo;
    }
}

public class Pelicula {
    LocalDate fechaEstreno;
    // ...

    public double getCosto() {
        return this.costo;
    }

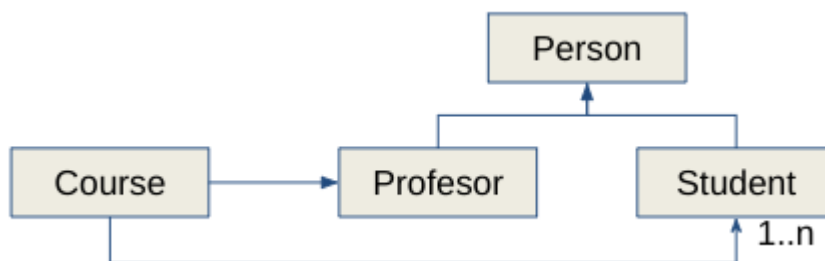
    public double calcularCargoExtraPorEstreno() {
        // Si la Película se estrenó 30 días antes de la fecha actual, retorna
        un cargo de 0$, caso contrario, retorna un cargo extra de 300$
        return (ChronoUnit.DAYS.between(this.fechaEstreno, LocalDate.now())) >
        30 ? 0 : 300;
    }
}
```

Ejercicio 3

Una plataforma de cursos on-line está preparando su sistema para hacer una actualización en su sistema de administración de cursos, alumnos y docentes. Los requerimientos iniciales del sistema fueron:

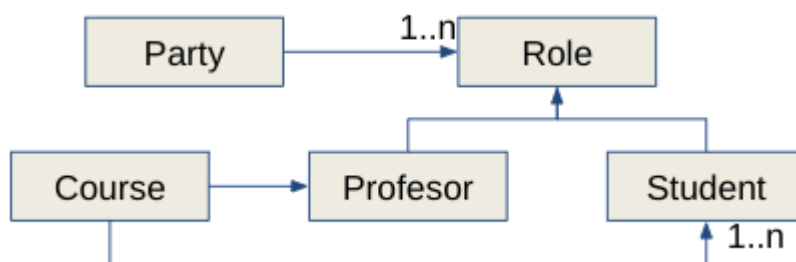
- El sistema permite registrar los cursos, los docentes y a los alumnos.
- Un Curso tiene un docente.
- Un Curso debe tener al menos un alumno.

La plataforma busca poder implementar otro tipo de cursos como talleres en donde los alumnos forman grupos de trabajo y se agrega la figura de mentor. Al mismo tiempo se requiere que una persona pueda ser profesor y mentor. El diagrama de clases simplificado del diseño original se presenta a continuación.



Antes de avanzar con la actualización, la empresa que desarrolló la plataforma propone hacer un refactoring que prepare el diseño para ser extendido en un siguiente paso mientras que implementa los requerimientos originales. El diseño resulta de aplicar el patrón Roles en donde una persona (Party) puede tener diferentes roles dentro de un modelo.

El diagrama de clases simplificado del diseño resultante del refactoring se presenta a continuación.



La plataforma le solicita que audite el diseño propuesto demostrando si la transformación propuesta es o no un refactoring.



Ejercicio 4

Considere el código fuente de la clase `Student` que se presenta a continuación.

```
import java.util.Vector;
import java.util.Iterator;

class Student {
    class ClassTaken {
        String name;
        Boolean done;
        int grade;
        public ClassTaken(String aName) {
            name = aName;
            done = false;
            grade = 0;
        }
    }

    String name;
    private Vector<ClassTaken> classes; // Plan de estudio
    String id;

    public Student(String newName, String ident) {
        name = newName;
        id = ident;
        classes = new Vector<ClassTaken>();
    }

    void addClass(String name) {
        classes.add(new ClassTaken(name));
    }

    void addDoneClass(String name) {
        ClassTaken aClass = new ClassTaken(name);
        aClass.done = true;
        classes.add(aClass);
    }

    void addGradedClass(String name, int grade) {
        ClassTaken aClass = new ClassTaken(name);
```



```
aClass.done = true;
aClass.grade = grade;
classes.add(aClass);
}

float finalsOverTaken() {
    int finals = 0;
    int done = 0;
    ClassTaken taken;
    Iterator it = classes.iterator();
    while(it.hasNext()){
        taken = (ClassTaken) it.next();
        if (taken.done)
            done++;
        if (taken.grade >= 6)
            finals++;
    }
    return (float) finals / done ;
}

boolean fitForScholarship() {
    int tally = 0;
    Iterator it = classes.iterator();
    ClassTaken taken;
    while (it.hasNext()) {
        taken = (ClassTaken) it.next();
        if (taken.grade >= 4)
            tally++;
    }
    return ((float) tally / classes.size() > 0.5) &&
        (this.finalsOverTaken() > 0.66);
}
}
```

Tareas:

1. Documente el código con un diagrama de clases UML
2. Escriba las reglas que hacen que un alumno esté en condiciones de recibir una beca, implementadas por el método fitForScholarship().
3. Desde el código original se implementaron algunas transformaciones:

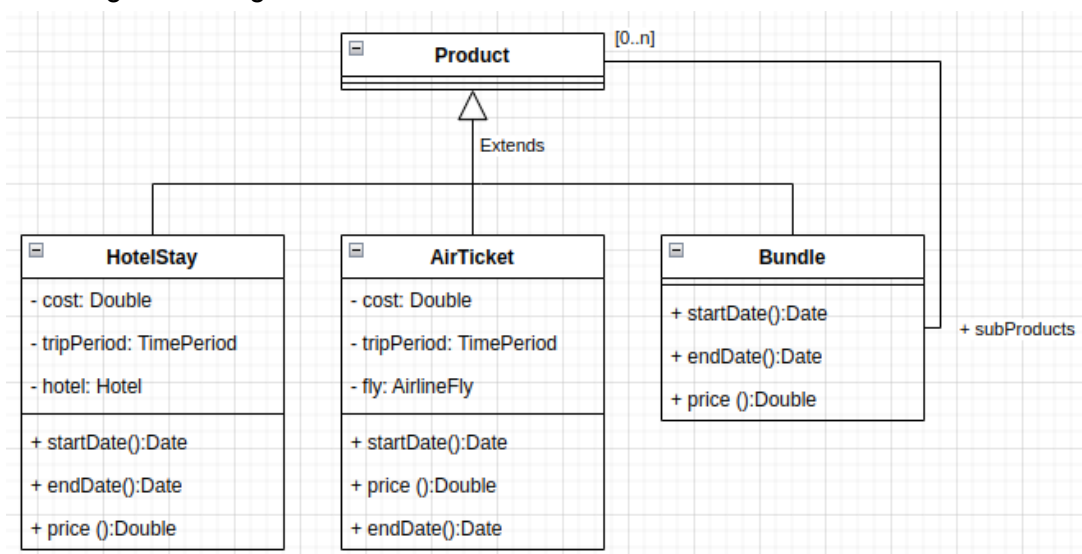


Se eliminó constante (hardcoded)	<pre>this.finalsOverTaken()>0.66</pre> se cambió a <pre>this.finalsOverTaken()>fitRatio()</pre> y se implementó el método <pre>float fitRatio() { return 2/3; }</pre>
Se eliminó constante (hardcoded)	<pre>(float) tally / classes.size() >= 0.5</pre> se cambió por <pre>(float) tally / classes.size() >= finalsRatio()</pre> y se implementó el método <pre>float finalsRatio() { return 1/2; }</pre>
Cambio de nombre	Se renombró el método <code>finalsOverTaken()</code> a <code>goodGradesOverTaken()</code>

- Evalúe (analizando el código) si estos cambios pueden ser refactorings o no
- Compile el código del proyecto original y del refactorizado que se encuentra en el Moodle de ROO2_2025 y compare los resultados de correr el programa `StudentCmd` (original y refactorizado).

Ejercicio 5

Estudie el siguiente diagrama de clases:





HotelStay	startDate() { return tripPeriod.start; } endDate() { return tripPeriod.end; } price() { return tripPeriod.duration() * hotel.nightPrice() * hotel.discountRate(); }
AirTicket	startDate() { return tripPeriod.start; } endDate() { return tripPeriod.end; } price() { return fly.price() * fly.promotionRate(); }
Bundle	startDate() { return min(subProducts.startDate()); } endDate() { return max(subProducts.startDate()); } price() { return sum(subProducts.price()); }

Refactorizar de manera que cumpla con las siguientes condiciones:

- Se requiere eliminar la definición de variables duplicadas.
- Se requiere eliminar código repetido.
- Se requiere minimizar la cantidad de definiciones del método **price()**.



Ejercicio 6 - Notas

En el sistema de cálculo de notas de alumnos de una facultad se contaba con el siguiente método para calcular promedios de notas de los alumnos:

```
1 public double promedio(List<Integer> notas) {  
2     int suma = 0;  
3     for (Integer n : notas) {  
4         suma += n;  
5     }  
6     return suma / notas.size();  
7 }
```

Se evaluó que dicho método presentaba varios bad smells y se decidió aplicar refactorings para solucionarlos, produciendo el siguiente código:

```
1 public class Nota {  
2     private final int valor;  
3  
4     public Nota(int valor) {  
5         if (valor < 0 || valor > 10) {  
6             throw new IllegalArgumentException("Nota inválida");  
7         }  
8         this.valor = valor;  
9     }  
10  
11     public int getValor() {  
12         return valor;  
13     }  
14 }  
15  
16 public class EstadisticaNotas {  
17     public static double promedio(List<Nota> notas) {  
18         return notas.stream()  
19             .mapToInt(Nota::getValor)  
20             .average()  
21             .orElse(0.0);  
22     }  
23 }
```

Tareas:

1. Analice ambas versiones del código y determine si se realizó un correcto refactoring. Justifique.
2. Determine los bad smells que fueron detectados y qué refactorings se aplicaron para solucionarlos. ¿Ve algún otro bad smell que no se haya detectado?



3. ¿El nuevo código presenta bad smells? De ser así, indique cuáles y qué refactoring aplicaría para solucionarlos.
4. Describa el comportamiento observable de cada versión del código.

Ejercicio 7 - Precio final

Un sistema de venta de productos cuenta con el siguiente método:

```
1 public double calcularPrecioFinal(double precio, double descuento, double impuestos) {  
2     return (precio - (precio * descuento)) + impuestos;  
3 }
```

Responda:

1. Analice el código e indique qué precondiciones se deben cumplir para con los datos de entrada y qué postcondiciones se garantizan tras la finalización del método.
2. Identifique claramente los malos olores en el código.
3. Aplique un refactoring que mejore la legibilidad y que permita expresar más claramente las precondiciones y postcondiciones del código.

Ejercicio 8 - Reserva de habitaciones

En el sistema de gestión de un hotel se cuenta con el siguiente método:

```
1 public Reserva reservarHabitacion(Hotel hotel, Cliente cliente,  
2     LocalDate desde LocalDate hasta) {  
3     Habitacion habitacion = hotel.buscarDisponible(desde, hasta);  
4     Reserva reserva = new Reserva(habitacion, cliente, desde, hasta);  
5     hotel.registrarReserva(reserva);  
6     return reserva;  
7 }
```

Responda:

1. ¿Qué supuestos deben cumplirse para que el método funcione correctamente?
2. ¿Qué se puede asumir tras la ejecución correcta del método?
3. Existe un problema cuando se intenta reservar habitaciones para un día sin vacantes disponibles, ¿qué refactoring se podría aplicar para corregir esto?

Ejercicio 9 - Primos

```
1 public class PrimeGeneratorStream {
2
3     public int currentNumber;
4
5     public PrimeGeneratorStream() {
6         this.currentNumber = 1;
7     }
8     public int next() {
9         while (true) {
10             if (isPrime(this.currentNumber)) {
11                 int nextPrime = this.currentNumber;
12                 this.currentNumber++;
13                 return nextPrime;
14             }
15             this.currentNumber++;
16         }
17     }
18     private boolean isPrime(int number) {
19         if (number < 1)
20             return false;
21         for (int i = 1; i <= Math.sqrt(number); i++) {
22             if (number % i == 0) {
23                 return false;
24             }
25         }
26         return true;
27     }
28 }
```

- Un colega que revisó el código se dio cuenta que este código es poco eficiente dado que las instancias de esta clase repiten mucho de sus cálculos y para lograr mayor eficiencia se propone “memorizar” los números primos ya calculados para no tener que recalcularlos. Presente el código resultante.
- Un colega que revisó el código se dio cuenta que hay un error, pero no le ha dicho cuál es. ¿Puede detectar el error y corregirlo? ¿Considera que ese cambio es un refactoring?
- Dado el cambio que Ud. presentó para que la clase “recuerde” los números primos ya calculados. ¿Considera que es un refactoring?



- d) Un colega que revisó el código se dio cuenta de que la variable `currentNumber` no debería ser pública sino privada. ¿Qué condición o condiciones se deberían dar para que el cambio a privada se pueda considerar un refactoring?

Ejercicio 10

Analice los refactorings aplicados en los ejercicios 1, 2 y 5 (y capaz el 4). Para cada refactoring:

1. Indique precondiciones necesarias antes de aplicarlo.
2. Indique postcondiciones esperadas luego de aplicarlo.
3. Señale al menos un posible riesgo de aplicar el refactoring sin cumplir las precondiciones, y qué tipo de error concreto podría introducir (ej.: ruptura de encapsulamiento, fallo de compilación, etc.).

Si un mismo refactoring aparece en más de un ejercicio (ej. Rename Method), describalo una sola vez.

Ejercicio 11 - StremiOO

La plataforma "StremiOO" ofrece contenido multimedia y notifica a los usuarios sobre nuevos lanzamientos. Su notificador funciona de la siguiente manera:

```
1 public class Notificacion {
2     private String destinatario;
3     public Notificacion(String destinatario) { this.destinatario = destinatario; }
4     public void enviar(String tipo, String mensaje) {
5         if (tipo.equalsIgnoreCase("VIDEO")) {
6             System.out.println("Video: " + mensaje + " a " + destinatario);
7         } else if (tipo.equalsIgnoreCase("MUSICA")) {
8             System.out.println("Música: " + mensaje + " a " + destinatario);
9         }
10    }
11 }
12 //Ejemplo de uso:
13 Notificacion notificacion = new Notificacion("usuario@ejemplo.com");
14 notificacion.enviar("VIDEO", "Nuevo lanzamiento disponible");
```

Lo siguiente es el código resultante luego de aplicar modificaciones al código anterior:



```
1 public abstract class Notificacion {
2     protected String destinatario;
3     public Notificacion(String destinatario) { this.destinatario = destinatario; }
4     public abstract void enviar(String mensaje);
5 }
6 public class NotificacionVideo extends Notificacion {
7     public NotificacionVideo(String destinatario) { super(destinatario); }
8     public void enviar(String mensaje) {
9         System.out.println("Video: " + mensaje + " a " + destinatario);
10    }
11 }
12 public class NotificacionMusica extends Notificacion {
13     public NotificacionMusica(String destinatario) { super(destinatario); }
14     public void enviar(String mensaje) {
15         System.out.println("Música: " + mensaje + " a " + destinatario);
16    }
17 }
18 public class NotificacionPodcast extends Notificacion {
19     public NotificacionPodcast(String destinatario) { super(destinatario); }
20     public void enviar(String mensaje) {
21         System.out.println("Podcast: " + mensaje + " a " + destinatario);
22    }
23 }
24 //Ejemplo de uso:
25 Notificacion notificacion = new NotificacionVideo("usuario@ejemplo.com");
26 notificacion.enviar("Nuevo lanzamiento disponible");
```

Tareas:

1. Realice el diagrama de clases UML del código luego de los cambios realizados.
2. Analizando el código modificado, ¿Considera que todas las modificaciones realizadas corresponden a refactorizaciones? Fundamente su respuesta indicando:
 - a. Las precondiciones requeridas por el refactoring, y si en este caso se cumplen o no.
 - b. Las postcondiciones requeridas por el refactoring, y si en este caso se cumplen o no.

Ejercicio 12: Triángulo Rectángulo

Sea el siguiente código:



```
1  public class TrianguloRectangulo {
2      private int b; // Base
3      private int a; // Altura
4
5      public TrianguloRectangulo(int valorB, int valorA) {
6          this.b = valorB;
7          this.a = valorA;
8      }
9
10     public int getBase() {
11         return this.b;
12     }
13     public int getAltura() {
14         return this.a;
15     }
16
17     public double calcularArea(double base) {
18         double area = this.a * this.b / 2;
19         return area;
20     }
21 }
```

Tareas

1.- A partir del código proporcionado, completa los espacios en blanco de la siguiente tabla indicando según corresponda, la(s) línea(s) de código donde aparece el bad smell, el nombre del mismo, justificación que explique por qué es un bad smell y qué técnica de refactoring se emplea para resolverlos.

Bad smell	#Línea(s)	Justificación	Refactoring
Uncommunicative Name			
	#18		
			Remove Parameter

2.- Muestre el código resultante luego de aplicar los refactorings indicados.